

UNIVERSITY COLLEGE OF ENGINEERING NARASARAOPET
JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY, KAKINADA



A Project Report on
Network Anomaly Detection using Equilibrium Optimized Feature Selection

Team Members:

22031A0547	Shaik Yasmeen
22031A0554	Vajja Prasanth Kumar
22031A0530	Narala Bhanu Prakash
22031A0557	Yarlagadda Jahnavi
22031A0538	Ravada Yogendar Rao

Under the esteemed guidance of

Dr. G. Madhavi

Associate Professor and HOD (i/c)

TABLE OF CONTENTS

S.NO	CHAPTER NAME	PAGE NO
1.	INTRODUCTION	
	1.1 INTRODUCTION	
	1.2 SIGNIFICANCE OF THE PROJECT	
	1.3 SUMMARY	
2.	LITERATURE SURVEY	
	2.1 LITERATURE SURVEY	
	2.2 PROBLEM STATEMENT	
	2.3 SUMMARY	
3.	SYSTEM ANALYSIS	
	3.1 EXISTING SYSTEM	
	3.1.1 DRAWBACKS OF EXISTING SYSTEM	
	3.2 PROPOSED SYSTEM	
	3.2.1 ADVANTAGES OF PROPOSED SYSTEMS	
	3.2.2 OVERCOMING THE DRAWBACKS OF THE EXISTING SYSTEM	
	3.3 DATASET DESCRIPTION	
	3.4 SUMMARY	

4.	SOFTWARE REQUIREMENTS SPECIFICATION	
	4.1 HARDWARE REQUIREMENTS	
	4.2 SOFTWARE REQUIREMENTS	
	4.3 FUNCTIONAL REQUIREMENTS	
	4.4 NON-FUNCTIONAL REQUIREMENTS	
	4.5 SUMMARY	
5.	SYSTEM DESIGN	
	5.1 SYSTEM ARCHITECTURE	
	5.2 MODULES	
	5.3 UML DIAGRAMS	
	5.3.1 USE CASE DIAGRAM	
	5.3.2 SEQUENCE DIAGRAM	
	5.3.3 ACTIVITY DIAGRAM	
	5.4 SUMMARY	

ABSTRACT

Network anomaly detection plays a crucial role in securing modern communication networks by identifying malicious and abnormal activities within network traffic. With the increasing complexity and volume of network data, traditional anomaly detection systems face challenges in handling high-dimensional features, class imbalance, and evolving attack patterns. Existing anomaly detection approaches commonly rely on the KDDCup99 dataset and employ traditional machine learning and ensemble-based classifiers such as Naive Bayes, Isolation Forest, Support Vector Machines (SVM), XGBoost, and Light Gradient Boosting Machine (LightGBM). Although these methods provide reasonable detection performance, they largely depend on manually engineered features and are limited in capturing complex non-linear patterns, and the outdated nature of the KDDCup99 dataset further reduces their effectiveness in representing modern network traffic behavior. To address these limitations, this project proposes an optimized anomaly detection framework using the UNSW-NB15 dataset, which reflects contemporary network traffic characteristics and attack scenarios. The proposed system integrates Equilibrium Optimizer based feature selection to identify the most relevant network attributes and reduce dimensionality, followed by stacked autoencoder based representation learning to extract compact and meaningful latent features. To mitigate class imbalance, the Synthetic Minority Over-sampling Technique (SMOTE) is applied to balance normal and attack samples, and the processed feature representations are classified using ensemble learning models, namely Extra Trees and CatBoost. Extra Trees is employed to provide fast and robust classification by reducing overfitting through increased randomness, while CatBoost is used to improve detection accuracy by effectively learning complex and imbalanced network traffic patterns. The proposed anomaly detection framework formulates the detection task as a binary classification problem, distinguishing normal network traffic from attack traffic. The performance of the proposed system is evaluated using standard metrics including accuracy, precision, recall, F1-score, confusion matrices, and ROC curves. Overall, the proposed framework provides an efficient, scalable, and practical solution for real-world network anomaly detection.

LIST OF FIGURES

S.NO	FIG NO	DESCRIPTION	PAGE NO
1.	5.1	SYSTEM ARCHITECTURE	
2.	5.3.1	USE CASE DIAGRAM	
3.	5.3.2	SEQUENCE DIAGRAM	
4.	5.3.3	ACTIVITY DIAGRAM	

CHAPTER 1

INTRODUCTION

1.1 INTRODUCTION

Anomaly detection is a critical area of research in the field of data science and cybersecurity, focusing on the identification of unusual patterns or behaviors that deviate significantly from normal system activity. In network environments, anomalies often indicate malicious activities such as intrusions, attacks, or unauthorized access attempts. With the rapid growth of networked systems and internet based services, protecting networks from increasingly complex cyber threats has become a major challenge.

Traditional security mechanisms, such as rule based anomaly detection systems, are limited in their ability to detect new and evolving attacks. As a result, machine learning-based anomaly detection techniques have gained significant attention due to their capability to analyze large volumes of network traffic and identify hidden patterns. However, conventional machine learning approaches often rely on handcrafted features, suffer from class imbalance, and struggle with high-dimensional data, leading to reduced detection accuracy and increased false positives.

To address these challenges, this project focuses on developing an efficient network anomaly detection system using advanced machine learning and optimization techniques on the UNSW-NB15 dataset, a benchmark dataset for network anomaly detection. The proposed approach integrates feature selection using the Equilibrium Optimizer (EO) to identify the most relevant features, deep representation learning through a Stacked Autoencoder (SAE) to extract robust latent representations, and ensemble classifiers such as Extra Trees and CatBoost to enhance detection performance. Extra Trees provides fast training and strong generalization by introducing high randomness in tree construction, making it suitable for large-scale network traffic data, while CatBoost effectively captures complex patterns and handles class imbalance, thereby improving anomaly detection performance. The system is evaluated using standard performance metrics such as accuracy, precision, recall and F1-score to assess detection performance. By using this unified learning pipeline, the system effectively distinguishes between normal and anomalous network traffic, and improves overall network security.

1.2 SIGNIFICANCE OF THE PROJECT

The significance of this project lies in its ability to enhance network security by providing an effective and intelligent anomaly detection solution. As cyber-attacks continue to grow in frequency and complexity, traditional security mechanisms are often insufficient to detect novel and sophisticated threats.

This project addresses these challenges by applying advanced machine learning and optimization techniques to accurately identify anomalous network behavior. By integrating optimized feature selection, deep representation learning, and robust ensemble classifiers into a unified framework, the proposed system improves detection accuracy. The use of modern anomaly detection datasets and scalable models ensures practical applicability in real-world network environments. Overall, this project contributes to the development of reliable and efficient anomaly detection systems that support proactive threat detection and strengthen the security of networked systems.

1.3 SUMMARY

This chapter introduced the concept of anomaly detection and its importance in securing modern network environments. It discussed the challenges faced by traditional anomaly detection systems, such as their inability to detect complex and evolving cyber threats, reliance on handcrafted features, and high false positive rates. The need for advanced machine learning-based approaches to improve detection accuracy and reliability was highlighted. The chapter also emphasized the significance of the proposed anomaly detection system, which integrates optimization-based feature selection, deep representation learning, and ensemble classification techniques. By leveraging modern datasets and scalable models, the proposed approach aims to accurately distinguish between normal and anomalous network traffic, reduce false alarms, and enhance overall network security.

CHAPTER 2

LITERATURE SURVEY

2.1 LITERATURE SURVEY

[1] U. Fiore, F. Palmieri, A. Castiglione, and A. De Santis, “Network anomaly detection with the restricted Boltzmann machine,”

This work proposes a deep learning based network anomaly detection approach using Restricted Boltzmann Machines. The model learns latent representations from network traffic data, enabling the identification of hidden attack patterns that traditional rule-based systems fail to detect. The study demonstrates the effectiveness of representation learning in improving detection accuracy.

[2] V. Chandola A. Banerjee, and V. Kumar Anomaly detection, ACM Computing Surveys, 2009.

This survey presents a comprehensive overview of anomaly detection techniques, including statistical, proximity-based, and machine learning approaches. It discusses applications across multiple domains and highlights major challenges such as limited labeled data, high false alarm rates, and the need for adaptive detection mechanisms.

[3] D.E.Difallah, P. Cudre-Mauroux, and S.A.McKenna,“Scalable anomaly detection for smart city infrastructure networks,”

This study introduces scalable anomaly detection solutions designed for smart city network infrastructures. Experimental evaluation on large-scale traffic datasets reveals that many existing systems lack generalization and are effective only for specific anomaly types.

[4] E. Anceaume, Y. Busnel, E. L. Merrer, R. Ludinard, J. L. Marchand, and B. Sericola, “Anomaly characterization in large scale networks,”

The authors focus on anomaly characterization in large-scale distributed networks using statistical monitoring techniques. The work emphasizes scalability, reliability, and fault tolerance, highlighting the need for resilient detection systems under continuous network growth.

[5] M. Ahmed, A. N. Mahmood, and J. Hu, “A survey of network anomaly detection techniques,”

This survey compares statistical, rule-based, and machine learning techniques for network anomaly detection. The study identifies the trade-off between false positives and false negatives as a persistent limitation in existing systems.

[6] M. Usama, J. Qadir, A. Raza, H. Arif, K. A. Yau, Y. Elkhatib, A. Hussain, and A. Al-Fuqaha, “Unsupervised machine learning for networking: Techniques, applications and research challenges,”

This work reviews unsupervised machine learning approaches such as clustering and density-based methods for anomaly detection. The authors demonstrate the ability of these techniques to detect previously unseen attacks without requiring labeled data.

[7] K. Fotiadou, T.-H. Velivassaki, A. Voulkidis, D. Skias, S. Tsekeridou, and T. Zahariadis, “Network traffic anomaly detection via deep learning,”

This paper proposes a deep learning based encoder–decoder architecture combining convolutional and recurrent neural networks. The model effectively captures spatial and temporal dependencies in network traffic, leading to improved anomaly detection performance.

[8] M. A. Umer, K. N. Junejo, M. T. Jilani, and A. P. Mathur, “Machine learning for intrusion detection in industrial control systems: Applications, challenges, and recommendations,”

The study reviews supervised, unsupervised, and semi-supervised intrusion detection approaches for industrial control systems. It emphasizes the impact of feature quality and dataset imbalance on detection performance.

[9] A. A. Jihado and A. S. Girsang, “Hybrid deep learning network intrusion detection system based on convolutional neural network and bidirectional long short-term memory,”

This work evaluates multiple machine learning and deep learning models for intrusion detection. The study highlights the importance of preprocessing and feature selection in improving classification accuracy.

[10] S.A.Jebur, K.A.Hussein, H.K.Hoomod, L.Alzubaidi and J.Santamaría, “Review on deep learning approaches for anomaly event detection in video surveillance,”

This review discusses various deep learning models used for anomaly detection, including convolutional and recurrent neural networks. The study demonstrates that hybrid deep learning architectures outperform traditional classifiers.

[11] S. Naseer, Y. Saleem, S. Khalid, M. K. Bashir, J. Han, M. M. Iqbal, and K. Han, “Enhanced network anomaly detection based on deep neural networks,”

This paper proposes a convolutional neural network based network traffic classification framework. Experimental results show that deep learning models effectively handle high-dimensional traffic features.

[12] N. Kumar and S. Sharma, “A hybrid modified deep learning architecture for intrusion detection system with optimal feature selection,”

The authors propose a hybrid intrusion detection architecture integrating feature selection with deep learning. The approach improves detection accuracy while reducing computational complexity.

[13] S.Hajj, R. El Sibai, J. B. Abdo, J. Demerjian, A. Makhoul, and C. Guyeux, “Anomaly-based intrusion detection systems: The requirements, methods, measurements, and datasets,”

This study presents a semi-supervised anomaly detection framework that combines labeled and unlabeled data. The approach improves detection performance in scenarios with limited labeled datasets.

[14] L. I. Khalaf, B. Alhamadani, O. A. Ismael, A. A. Radhi, S. R. Ahmed, and S. Algburi, “Deep learning-based anomaly detection in network traffic for cyber threat identification,”

This work evaluates convolutional neural networks, recurrent neural networks, autoencoders, and generative adversarial networks for network anomaly detection. The results confirm that autoencoder-based feature learning significantly enhances detection accuracy.

[15] S. Gunupusala and S. C. Kaila, “Multi-class network anomaly detection using machine learning techniques,”

The authors propose a multi-class anomaly detection system using feature reduction techniques. The system improves efficiency and classification performance by eliminating redundant features.

[16] K. Lu, “Network anomaly traffic analysis,”

This paper presents a hybrid anomaly detection framework combining traditional machine learning and deep learning techniques to improve robustness and detection reliability.

[17] A. Alfardus and D.B. Rawat, “Machine learning-based anomaly detection for securing in-vehicle networks,”

The proposed system detects anomalies in vehicular communication networks. The study demonstrates effective real-time anomaly detection capabilities.

[18] V. Takale, A. Patil, A. Lonikar, A. Shinde, and P. V. Rupnar, “SecureNet: Network intrusion detection using machine learning and deep learning techniques,”

This work introduces SecureNet, a hybrid intrusion detection system combining long short-term memory and k-nearest neighbor classifiers. The system achieves enhanced detection accuracy through hybrid learning.

[19] S. H. Rafique, A. Abdallah, N. S. Musa, and T. Murugan, “Machine learning and deep learning techniques for Internet of Things network anomaly detection—Current research trends,”

This survey analyzes machine learning and deep learning approaches for IoT anomaly detection. It emphasizes scalability and adaptability to emerging cyber threats.

[20] M. Mynuddin, S. U. Khan, Z. U. Chowdhury, F. Islam, M. J. Islam, M. I. Hossain, and D. M. A. Ahad, “Automatic network intrusion detection system using machine learning and deep learning,”

The authors propose a convolutional neural network and bidirectional long short-term memory based intrusion detection model. The system demonstrates improved detection accuracy compared to conventional approaches.

[21] S. Marappan and H. Marappan, “Deep learning-based intelligent algorithms for effective transmission authentication and anomaly identification in vehicular networks,”

This study introduces an attention-based bidirectional long short-term memory framework for vehicular networks. The model achieves high accuracy with low detection latency.

[22] S. Ola-Obaado and M. A. Suleiman, “Anomaly-based network intrusion detection using transfer learning,”

The authors integrate transfer learning with SMOTE to address class imbalance. The approach improves detection of minority attack classes.

[23] L. Lahesoo, U. Do, R. Carnier, and K. Fukuda, “SIURU: A framework for machine learning based anomaly detection in IoT network traffic,”

This framework applies unsupervised learning for IoT traffic anomaly detection. A limitation observed is reduced effectiveness against previously unseen attacks.

[24] M. Rele and D. Patil, “Intrusive detection techniques utilizing machine learning, deep learning, and anomaly-based approaches,”

This study highlights the benefits of combining machine learning and deep learning models for improved intrusion detection accuracy.

[25] T. Ahmad and D. Truscan, “Efficient early anomaly detection of network security attacks using deep learning,”

The authors propose a one-dimensional convolutional neural network based intrusion detection system that enables early detection with reduced computational overhead.

[26] M. A. Siddiqui, M. Kalra, and C. Rama Krishna, “Anomaly detection for IoT-enabled kitchen area network using machine learning,”

This work presents a distance-based k-nearest neighbor anomaly detection approach that effectively identifies anomalous IoT traffic behavior.

[27] Y. Akhiat, K. Touchanti, A. Zinedine, and M. Chahhou, “IDS-EFS: Ensemble feature selection-based method for intrusion detection system,”

The proposed framework improves intrusion detection accuracy through optimized ensemble feature selection.

[28] M. Tavallaei, E. Bagheri, W. Lu, and A. A. Ghorbani, “A detailed analysis of the KDD CUP 99 data set,”

This study identifies redundancy and bias in the KDD Cup 99 dataset, highlighting limitations that affect intrusion detection reliability.

[29] G. Fernandes, J. J. P. C. Rodrigues, L. F. Carvalho, J. F. Al-Muhtadi, and M. L. Proença, “A comprehensive survey on network anomaly detection,”

This comprehensive survey reviews anomaly detection techniques, datasets, and evaluation metrics, identifying gaps in benchmarking practices.

[30] F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation forest,”

The Isolation Forest algorithm efficiently detects anomalies in high-dimensional datasets with low computational cost.

[31] H.Chen, S.Hu, R.Hua and X.Zhao, “Improved naive Bayes classification algorithm for traffic risk management,”

This work enhances Naïve Bayes classification for traffic risk management, improving stability and efficiency.

[32] M. Al-kasassbeh, M. A. Abbadi, and A. M. Al-Bustanji, “Light GBM algorithm for malware detection,”

The LightGBM-based model achieves high accuracy with fast training time on security datasets.

[33] D. M. Abdullah and A. M. Abdulazeez, “Machine learning applications based on SVM classification review,”

This review outlines the strengths and limitations of support vector machine classifiers across multiple applications.

[34] M. Nemeth, D. Borkin, and G. Michalconok, “The comparison of machine learning methods XGBoost and LightGBM to predict energy development,”

The study highlights trade-offs between prediction performance and computational efficiency of boosting algorithms.

[35] O. Rainio, J. Teuho, and R. Klén, “Evaluation metrics and statistical tests for machine learning,”

This work emphasizes the importance of selecting appropriate evaluation metrics for reliable model assessment.

[36] M. Hossin and M. N. Sulaiman, “A review on evaluation metrics for data classification evaluations,”

The authors discuss evaluation metrics for imbalanced datasets and the limitations of accuracy-based assessment.

[37] G. Varoquaux and O. Colliot, “Evaluating machine learning models and their diagnostic value,”

This study emphasizes reproducibility and reliable validation practices in machine learning experiments.

[38] M. A. Aslam, F. Murtaza, M. E. U. Haq, A. Yasin, and M. A. Azam, “A human-centered approach to academic performance prediction using personality factors in educational AI,”

The authors highlight the importance of systematic and transparent evaluation procedures.

[39] A.Halbouni, T.S.Gunawan, M.H.Habaebi, M.Halbouni, M.Kartiwi, and R. Ahmad, “CNN-LSTM: Hybrid deep neural network for network intrusion detection system,”

The proposed model captures spatial and temporal features, improving detection of complex attack patterns.

[40] C. Xu, J. Shen, X. Du, and F. Zhang, “An intrusion detection system using a deep neural network with gated recurrent units,”

This work demonstrates that GRU-based deep neural networks effectively model temporal dependencies in network traffic, achieving higher detection accuracy

2.2 PROBLEM STATEMENT

The rapid expansion of computer networks and the increasing sophistication of cyber attacks have made effective network anomaly detection a critical requirement for modern communication systems. Traditional anomaly detection systems largely rely on rule-based mechanisms or conventional machine learning techniques that depend on manually engineered features and outdated benchmark datasets such as KDD Cup 99. These approaches struggle to handle high-dimensional network traffic data, suffer from feature redundancy, and exhibit limited adaptability to evolving and previously unseen attack patterns. Additionally, network traffic datasets are often highly imbalanced, leading to biased detection performance and increased false negatives for minority attack classes. Many existing detection models also face challenges related to scalability, robustness, and generalization, restricting their effectiveness in real-world, large-scale network environments. Consequently, there is a need for an optimized and adaptive anomaly detection approach that can efficiently process complex network traffic data, address class imbalance, and provide accurate and scalable detection of anomalous activities in contemporary network settings.

2.3 SUMMARY

This chapter discussed various existing works related to network anomaly detection. It reviewed the different techniques and technologies used in developing intrusion detection systems, along with their advantages and limitations. The study showed that traditional and existing approaches face challenges such as feature redundancy, class imbalance, outdated datasets, and limited adaptability to evolving attack patterns. Although several systems have been proposed in the literature, each has certain drawbacks that affect overall detection accuracy and scalability.

CHAPTER 3

SYSTEM ANALYSIS

3.1 EXISTING SYSTEM

The existing system for network anomaly detection is based on traditional machine learning approaches applied to benchmark anomaly detection datasets such as KDDCup'99. The primary objective of the system is to classify network traffic into Normal and Attack categories using handcrafted network features. Network traffic data is collected from publicly available datasets and undergoes basic preprocessing steps, including handling missing values through mean substitution, removal of duplicate records, label encoding of categorical attributes such as protocol type, service, and flag, and feature normalization using z-score standardization. Since the dataset is highly imbalanced, the Synthetic Minority Over-sampling Technique (SMOTE) is applied to balance the class distribution.

For feature handling, the system either utilizes all available features directly or applies Principal Component Analysis (PCA) for dimensionality reduction, which emphasizes variance preservation rather than feature relevance. After preprocessing, multiple individual machine learning models are trained and evaluated independently, including LightGBM, XGBoost, Support Vector Machine (SVM), Naïve Bayes, and Isolation Forest, covering both supervised and unsupervised learning approaches. The system generates a binary classification output indicating normal or anomalous traffic, and model performance is assessed using standard evaluation metrics such as accuracy, precision, recall, F1-score, and ROC curves. The results mainly focus on comparing individual model performance, where ensemble boosting models achieve better results than simpler classifiers, while unsupervised models such as Isolation Forest demonstrate weak generalization capability.

LightGBM is a gradient boosting-based ensemble model that efficiently learns complex feature interactions from network traffic data. Its tree-based structure enables fast training and improved detection accuracy, making it one of the better-performing models in the existing system.

XGBoost is an optimized boosting algorithm that builds decision trees sequentially to minimize classification errors. In the existing system, it effectively captures non-linear relationships in network features and provides anomaly detection performance through regularized learning.

Support Vector Machine (SVM) is a supervised classification algorithm that separates normal and anomalous traffic using an optimal decision boundary. Although SVM performs well on smaller datasets, its computational complexity limits scalability for large and high-dimensional network traffic data.

Naive Bayes is a probabilistic classifier based on Bayes theorem and conditional independence assumptions. It offers fast execution and low computational cost but shows reduced accuracy when handling complex and correlated network traffic features.

Isolation Forest is an unsupervised anomaly detection model that identifies anomalies by isolating data points through random partitioning. While it does not require labeled data, its generalization capability is limited for sophisticated intrusion patterns.

3.1.1 DRAWBACKS OF EXISTING SYSTEM

1. Reliance on Handcrafted Features

The existing system depends on manually extracted network features. Such features may not fully capture complex and hidden patterns present in network traffic, which can limit anomaly detection performance.

2. Lack of Intelligent Feature Selection

The system either uses all available features or applies PCA, which reduces dimensionality based on variance rather than feature importance. This may result in the inclusion of less relevant features for anomaly detection.

3. Limited Capability in Handling Complex Attack Patterns

Traditional machine learning models used in the system have limited ability to model highly non-linear and complex relationships in network traffic data.

4. Class Imbalance Issue

Although SMOTE is applied, class imbalance remains a challenge in anomaly detection datasets, which can affect the learning process and lead to biased predictions.

5. Standalone Model-Based Detection

Each machine learning model is trained and evaluated independently. The absence of an integrated or hybrid detection framework limits robustness and overall detection coverage.

6. Weak Performance of Unsupervised Models

Unsupervised techniques such as Isolation Forest show lower detection accuracy compared to supervised models, reducing their effectiveness in identifying anomalies.

7. Scalability Concerns

Some traditional models, especially SVM, may face scalability issues when dealing with large-scale or high-dimensional network traffic data.

8. Lacks Adaptability to Evolving Attacks

Reliance on the outdated KDDCup'99 dataset limits the system's ability to detect modern and newly emerging attack patterns.

3.2 PROPOSED SYSTEM

The proposed system is designed to improve network anomaly detection by integrating optimization-based feature selection, deep representation learning, data balancing, and robust ensemble classifiers into a unified detection pipeline. The system is implemented and evaluated using the UNSW-NB15 dataset, which contains both normal and malicious network traffic with modern attack behaviors. The dataset includes numerical and categorical features such as protocol type, service, and flag, and the target variable represents a binary classification indicating normal or attack traffic. The dataset is divided into training and testing sets to ensure unbiased evaluation.

Initially, the UNSW-NB15 data undergoes preprocessing, which includes handling missing values, encoding categorical attributes, and feature scaling using standard scaler. After preprocessing, the Equilibrium Optimizer is applied for feature selection, where the most relevant features are selected while eliminating redundant and irrelevant ones. This step reduces dimensionality and improves computational efficiency. The selected features are then passed to an Autoencoder, which performs representation learning by learning compact and non-linear feature representations that capture hidden patterns in network traffic. To address the class imbalance inherent in intrusion detection data, SMOTE is applied only to the training data, ensuring balanced learning without data leakage.

The transformed and balanced feature representations are finally classified using ensemble learning models, namely Extra Trees and CatBoost. Extra Trees improves robustness by introducing randomness in feature selection and split points, while CatBoost efficiently handles feature interactions and reduces prediction bias. The overall model flow follows the sequence: preprocessing, feature selection, representation learning, data balancing, and classification. The system outputs a binary decision indicating whether the network traffic is normal or anomalous.

Performance is evaluated using metrics such as accuracy, precision, recall, F1-score, and balanced accuracy, demonstrating improved detection capability compared to traditional standalone models.

Extra Trees Classifier

The Extra Trees classifier will be employed to enhance diversity in ensemble learning. It will introduce additional randomness in feature selection and split points, leading to improved generalization performance. This classifier will efficiently handle high-dimensional data and complex decision boundaries.

CatBoost Classifier

The CatBoost classifier will be utilized as an advanced boosting-based ensemble model for anomaly detection. It will handle categorical feature behavior effectively after encoding and will reduce prediction bias through ordered boosting. This model will contribute to improved classification accuracy and stability.

Equilibrium Optimizer (EO)

In the proposed anomaly detection system, the Equilibrium Optimizer (EO) is used for feature selection. EO is a meta-heuristic optimization algorithm that searches for an optimal subset of network traffic features by balancing exploration and exploitation.

In the code, EO evaluates different feature combinations using a fitness function based on classification performance and selects the most relevant features that contribute to anomaly detection. This reduces feature dimensionality, removes redundant and irrelevant attributes, and improves computational efficiency while maintaining high detection accuracy..

Autoencoder

The autoencoder will be utilized for deep feature learning and representation extraction. It will learn compact and meaningful latent features from the EO-selected input data, capturing complex and non-linear patterns in network traffic behavior. These encoded features will enhance the performance and generalization capability of the anomaly detection classifiers.

3.2.1 ADVANTAGES OF PROPOSED SYSTEM

1. Optimized Feature Selection

The use of the Equilibrium Optimizer enables effective selection of the most relevant features, reducing redundancy and improving learning efficiency compared to using all features or PCA.

2. Feature Representation Learning

The autoencoder learns compact and meaningful feature representations, allowing the system to capture complex and non-linear patterns in network traffic that handcrafted features may miss.

3. Detection Accuracy

By combining optimized features, deep representations, and ensemble classifiers, the proposed system achieves better anomaly detection accuracy and generalization.

4. Effective Handling of Class Imbalance

Applying SMOTE only on the training data balances class distribution without causing data leakage, leading to fair and reliable model evaluation.

5. Robust Ensemble Classification

Extra Trees and CatBoost improve robustness by handling feature variability, noise, and complex feature interactions more effectively than single traditional models.

6. Scalable and Efficient Framework

Tree-based ensemble models scale well with large and high-dimensional datasets, making the system suitable for real-world network environments.

7. Integrated Detection Pipeline

The proposed system follows a unified workflow from preprocessing to classification, unlike standalone model evaluation approaches, improving overall detection coverage.

8. Better Generalization to Modern Attacks

Using the UNSW-NB15 dataset and advanced learning techniques improves the system's ability to detect modern and diverse attack patterns

3.2.2 OVERCOMING DRAWBACKS OF THE EXISTING SYSTEM

1. Reduction of Handcrafted Feature Dependence

The proposed system uses an Autoencoder to automatically learn deep and non-linear feature representations, reducing reliance on manually engineered features.

2. Optimized Feature Selection

The Equilibrium Optimizer selects only the most relevant features, eliminating redundant and irrelevant attributes present in the existing system.

3. Improved Handling of High-Dimensional Data

Feature selection and representation learning together reduce dimensionality, improving learning efficiency and detection performance.

4. Effective Class Imbalance Handling

SMOTE is applied only on the training data, improving minority class learning while avoiding data leakage.

5. Integrated Detection Framework

Unlike standalone models in the existing system, the proposed approach integrates feature selection, representation learning, and classification into a unified pipeline.

6. Enhanced Classification Robustness

Ensemble classifiers such as Extra Trees and CatBoost improve robustness by handling noise, feature variability, and complex feature interactions.

7. Improved Scalability

Tree-based ensemble models scale well with large and high-dimensional datasets, making the system suitable for real-world network environments.

3.3 DATASET DESCRIPTION

The UNSW-NB15 dataset is a modern benchmark dataset widely used for network anomaly detection and intrusion detection research. It was generated by the Cyber Range Laboratory of the Australian Centre for Cyber Security (ACCS) using the IXIA PerfectStorm tool, which simulates realistic network traffic along with contemporary cyber-attack scenarios. The dataset was developed to overcome the limitations of older anomaly detection datasets by incorporating diverse attack types and detailed network flow characteristics that closely resemble real-world network environments.

Dataset Source

- **Repository:** Australian Centre for Cyber Security (ACCS) / Kaggle
- **Dataset Name:** UNSW-NB15 (University of New South Wales – Network-Based 2015)
- **Dataset URL:** <https://www.kaggle.com/datasets/mrwellsdavid/unswnb15>

Original Dataset and Predefined Dataset Version

The original UNSW-NB15 dataset contains 49 attributes, comprising 47 traffic-related features and 2 target variables (attack_cat and label). These attributes include low-level identifier features such as source and destination IP addresses and port numbers.

In this project, the predefined training and testing datasets are used, which consist of **45** columns, including:

- 43 input features
- 1 binary target variable (label)
- 1 auxiliary attribute (attack_cat, not used for classification)

The reduction from 49 to 45 attributes is due to the removal of high-cardinality identifier features (such as IP addresses and port numbers). These features are excluded because they are environment-specific, do not generalize well across different networks, and may lead to overfitting. The predefined dataset version is widely adopted in machine learning-based anomaly detection studies and is more suitable for building robust and generalizable models.

Dataset Characteristics

The UNSW-NB15 dataset exhibits class imbalance, with attack traffic dominating normal traffic in the predefined datasets. This reflects realistic network conditions and presents challenges for anomaly detection. The dataset's diverse attack behaviors and rich feature set make it suitable for machine learning-based anomaly detection.

Dataset Size and Train–Test Split

The dataset is provided in predefined training and testing sets, enabling reproducible experimentation and fair performance evaluation:

- **Training set:** 1,75,341 records
- **Testing set:** 82,332 records

Feature Set Description

Each network flow record in the predefined UNSW-NB15 dataset is represented using 43 input features that describe various aspects of network traffic behavior. These features are grouped into the following categories:

1. Basic Flow Features

These features describe the basic properties of a network connection, such as its duration, protocol, and current state.

Features: dur, proto, service, state

2. Packet-Based Features

These features represent packet count and data volume exchanged between the source and destination during a connection.

Features: spkts, dpkts, sbytes, dbytes

3. Time-Based Features

These features capture timing-related characteristics of network traffic, including packet intervals and TCP timing behavior.

Features: sinpkt, dinpkt, tcprtt, synack, ackdat

4. Traffic Load and Behavior Features

These features describe traffic intensity and variation in packet transmission from both source and destination.

Features: sload, dload, sjit, djit

5. Statistical and Connection Features

These features summarize historical connection patterns and statistical relationships between source, destination, and services.

Features:

ct_srv_src, ct_state_ttl, ct_dst_ltm, ct_src_dport_ltm,
ct_dst_sport_ltm, ct_dst_src_ltm, ct_src_ltm, ct_srv_dst

6. Application-Level and Binary Indicator Features

These features capture application-specific behavior and binary indicators related to protocol usage.

Features:

trans_depth, response_body_len, ct_flw_http_mthd,,ct_ftp_cmd, is_ftp_login, is_sm_ips_ports

Description of Features in the UNSW-NB15 Dataset

S.NO	FEATURE NAME	DESCRIPTION
1	id	Unique identifier for each network flow
2	dur	Duration of the connection (seconds)
3	proto	Transport protocol used (TCP, UDP, ICMP, etc.)
4	service	Network service type (HTTP, FTP, DNS, etc.)
5	state	Connection state (e.g., FIN, CON, INT)
6	spkts	Number of packets sent from source to destination
7	dpkts	Number of packets sent from destination to source
8	sbytes	Number of bytes sent from source to destination
9	dbytes	Number of bytes sent from destination to source
10	rate	Connection rate (packets per second)
11	sttl	Source to destination Time-To-Live (TTL) value
12	dttl	Destination to source Time-To-Live (TTL) value
13	sload	Source bits per second
14	dload	Destination bits per second
15	sloss	Number of packets lost from source
16	dloss	Number of packets lost from destination
17	sinpkt	Average time between packets from source
18	dinpkt	Average time between packets from destination
19	sjit	Source jitter (packet arrival variation)
20	djit	Destination jitter (packet arrival variation)
21	swin	TCP window size of source
22	stcpb	Source TCP base sequence number
23	dtcpb	Destination TCP base sequence number
24	dwin	TCP window size of destination
25	tcprtt	TCP round-trip time
26	synack	Time between SYN and SYN-ACK packets
27	ackdat	Time between SYN-ACK and ACK packets

28	smean	Mean packet size from source
29	dmean	Mean packet size from destination
30	trans_depth	Depth of HTTP transaction
31	response_body_len	Size of HTTP response body
32	ct_srv_src	Number of connections with same service and source
33	ct_state_ttl	Number of connections with same state and TTL
34	ct_dst_ltm	Number of connections to same destination in last time window
35	ct_src_dport_ltm	Connections from same source to same destination port
36	ct_dst_sport_ltm	Connections to same destination from same source port
37	ct_dst_src_ltm	Connections between same source and destination
38	is_ftp_login	Indicates FTP login success (binary)
39	ct_ftp_cmd	Number of FTP commands
40	ct_flw_http_mthd	Number of HTTP methods used in flow
41	ct_src_ltm	Connections from same source in last time window
42	ct_srv_dst	Connections to same service and destination
43	is_sm_ips_ports	Indicates same IPs and ports used (binary)
44	attack_cat	Attack category (e.g., DoS, Exploit, Fuzzers)
45	label	Target class: 0 = Normal, 1 = Attack

Target Variable (Binary Classification)

This project formulates the anomaly detection task as a binary classification problem. The label attribute is used as the target variable and is defined as:

0 – Normal network traffic

1 – Attack (Anomalous network traffic)

The objective is to accurately distinguish between normal and anomalous network flows using the extracted feature set.

3.4 SUMMARY

This chapter presents a detailed analysis of the existing and proposed network anomaly detection systems. It discusses the limitations of traditional machine learning approaches based on handcrafted features and standalone models. The chapter describes the proposed integrated framework using the UNSW-NB15 dataset with Equilibrium Optimizer–based feature selection, autoencoder-based representation learning, SMOTE, and ensemble classifiers. It also highlights how the proposed system improves detection accuracy, robustness, and scalability.

CHAPTER 4
SOFTWARE REQUIREMENTS SPECIFICATION

4.1 HARDWARE REQUIREMENTS

The proposed anomaly detection system is implemented using machine learning and deep learning techniques and does not require specialized hardware for basic execution. The system can be developed and evaluated either on a local machine or using cloud-based platforms such as Google Colab. The following hardware configuration is sufficient for effective implementation:

- **Processor:** Intel Core i5 or higher
- **Clock Speed:** 2.5 GHz or above
- **RAM:** Minimum 8 GB (16 GB recommended for improved performance)
- **Storage:** At least 20 GB of free disk space
- **Graphics Processor (Optional):** GPU support for faster autoencoder training (available through Google Colab)
- **System Architecture:** 64-bit operating system

4.2 SOFTWARE REQUIREMENTS

The proposed anomaly detection system is implemented using open-source software tools that support data preprocessing, optimization, deep learning, ensemble classification, and performance evaluation. The required software configuration is as follows:

- **Operating System:** Windows 10 / Linux / macOS (64-bit) or cloud-based platform
- **Programming Language:** Python 3.x
- **Development Environment:** Jupyter Notebook, Google Colab
- **Machine Learning Libraries:** Scikit-learn, CatBoost
- **Deep Learning Framework:** TensorFlow (Keras)
- **Class Imbalance Handling Tool:** Imbalanced-learn (SMOTE)
- **Data Processing Libraries:** Pandas, NumPy
- **Visualization Tools:** Matplotlib

4.3 FUNCTIONAL REQUIREMENTS

Functional Requirements define the specific functions and operations that the system must perform. They describe what the system does in terms of input processing, data handling, and output generation.

1. Network Traffic Data Collection

The proposed system is designed to load and manage network traffic data from the UNSW-NB15 dataset for anomaly detection. It supports structured datasets containing both normal and attack traffic records and ensures proper separation of training and testing data for reliable model evaluation.

2. Data Preprocessing and Transformation

The framework facilitates preprocessing of raw network traffic data by handling missing values, encoding categorical attributes, and feature scaling. This process improves data consistency and prepares the dataset for effective feature selection and machine learning model training.

3. Feature Selection using Equilibrium Optimizer

The methodology incorporates the Equilibrium Optimizer to identify the most significant network traffic features. This reduces dimensionality, removes redundant attributes, and improves computational efficiency while preserving anomaly detection performance.

4. Feature Learning using an Autoencoder

The model is designed to extract compact latent feature representations using a stacked autoencoder. This enables effective learning of hidden patterns in network traffic data and enhances the performance of classification models.

5. Class Imbalance Handling

The solution ensures balanced class distribution through the application of the Synthetic Minority Over-sampling Technique. This improves detection accuracy for minority attack classes and reduces bias in classification results.

6. Anomaly Classification

The proposed system performs anomaly detection using machine learning classifiers such as Extra Trees and CatBoost. These models accurately differentiate between normal and anomalous network traffic.

7. Performance Evaluation and Validation

The architecture allows evaluation of model performance using metrics including accuracy, balanced accuracy, precision, recall and F1-score. This assessment validates the effectiveness and reliability of the anomaly detection framework.

8. Result Visualization and Prediction

The application is capable of generating visual performance outputs such as confusion matrices, ROC curves, and precision–recall curves. It also supports anomaly prediction on unseen network traffic data using the trained detection models.

4.4 NON FUNCTIONAL REQUIREMENTS

Non-Functional Requirements specify the quality attributes and performance constraints of the system. They describe how well the system performs its functions, including aspects such as accuracy, efficiency, scalability, and reliability.

1. Performance

The system shall provide efficient anomaly detection with minimal processing delay. Model training and prediction should be completed within acceptable computational time limits for large-scale network datasets.

2. Computational Efficiency

The framework emphasizes efficient computation by reducing feature dimensionality and employing fast ensemble learning models. This enables timely processing of large-scale network traffic data during both training and prediction phases.

3. Robustness to Class Imbalance

The system maintains stable detection performance even in the presence of imbalanced network traffic data. The application of oversampling techniques and balanced evaluation metrics prevents bias toward majority classes.

4. Scalability

The system supports scalability to accommodate increasing network traffic volume and feature complexity. The modular design allows the framework to be extended to larger datasets without significant architectural changes.

5. Reproducibility

The system ensures reproducible experimental behavior and consistent detection performance through controlled preprocessing and fixed training configurations.

6. Reliability

The system provides consistent anomaly detection performance across repeated runs. Robust preprocessing, feature learning, and classification stages contribute to dependable and stable detection results.

4.5 SUMMARY

This chapter outlines the hardware and software requirements necessary for implementing the proposed anomaly detection system using machine learning and deep learning techniques. It describes the functional requirements covering data acquisition, preprocessing, feature selection using the Equilibrium Optimizer, deep feature extraction with autoencoders, class imbalance handling, anomaly classification, and performance evaluation. The non-functional requirements emphasize system accuracy, computational efficiency, robustness to class imbalance, scalability, reproducibility, and reliability. Overall, the chapter defines the technical and operational specifications required for effective and reliable anomaly detection.

CHAPTER 5

SYSTEM DESIGN

5.1 SYSTEM ARCHITECTURE

System architecture refers to the high-level structural design of a system that defines its components, their responsibilities, and how they interact with each other. It provides a blueprint for organizing hardware, software, and data flow within the system. System architecture helps in understanding system behavior, scalability, and overall design efficiency.

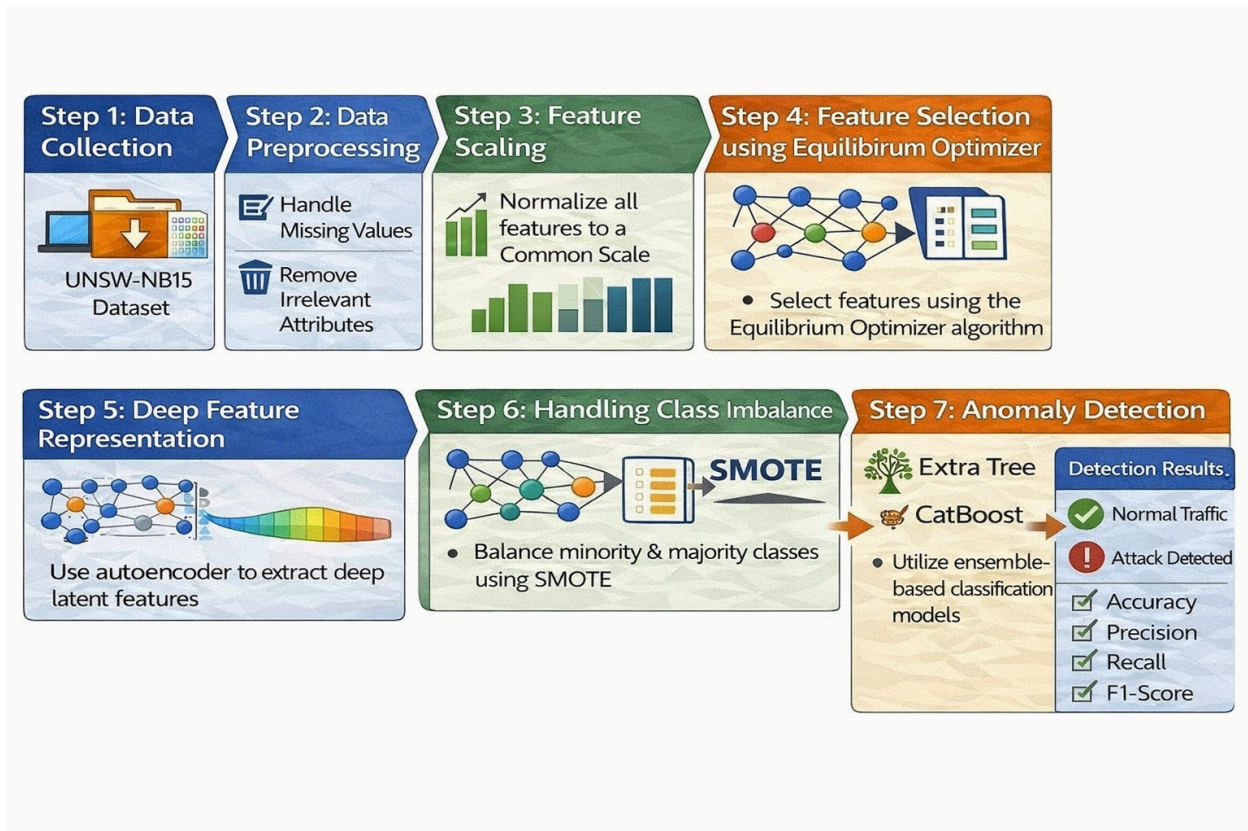


Fig 5.1 System Architecture

This diagram illustrates the complete workflow of the anomaly detection system using the UNSW-NB15 dataset. It begins with data collection, followed by data preprocessing, where missing values are handled, irrelevant attributes are removed, and categorical features are converted into numerical form using label encoding. Next, feature scaling is applied using StandardScaler to standardize all features to a common scale, ensuring stable training for the Autoencoder. An equilibrium-based feature selection method is then used to choose the most relevant subset of features. Deep feature representation is performed using a stacked autoencoder to extract meaningful latent features. To address class imbalance in network traffic data, SMOTE

is applied to balance the minority and majority classes. Finally, anomaly detection is carried out using ensemble-based classifiers such as Extra Trees and CatBoost, producing detection results that classify traffic as normal or attack and evaluate performance using metrics such as accuracy, precision, recall, and F1-score.

5.2 MODULES

1. Data Collection

The proposed system utilizes the UNSW-NB15 dataset for network anomaly detection. This dataset contains real-world network traffic records representing both normal behavior and various attack scenarios. Separate training and testing datasets are considered to support effective model training and unbiased performance evaluation.

2. Data Preprocessing

The collected network traffic data is preprocessed to improve data quality and consistency. Missing values are handled appropriately, and irrelevant attributes are removed. Categorical network attributes such as protocol, service, and state are transformed into numerical form using label encoding to enable computational processing.

3. Feature Scaling

All preprocessed features are standardized to a common scale using StandardScaler. This ensures uniform contribution during model learning, prevents features with larger numerical ranges from dominating, and improves training stability and convergence of subsequent models.

4. Feature Selection using Equilibrium Optimization

An optimization-based feature selection approach is applied to identify the most relevant subset of features. The method iteratively evaluates different feature combinations based on classification performance while maintaining class balance. This reduces dimensionality, eliminates redundant attributes, and enhances detection efficiency.

5. Deep Feature Representation

The selected feature subset is passed through a Stacked Autoencoder to obtain compact latent representations. This captures nonlinear relationships and hidden patterns present in network traffic, providing more informative features for anomaly classification.

6. Class Imbalance Handling

Class imbalance between normal and attack samples is addressed using SMOTE. Synthetic samples are generated to balance the minority class, improving the system's ability to detect rare and sophisticated attack patterns.

7. Anomaly Detection

The balanced deep feature representations are used to train ensemble-based classifiers, including Extra Trees and CatBoost. These models learn complex patterns in network traffic and collaboratively identify decision boundaries that distinguish normal traffic from anomalous behavior.

8. Performance Evaluation

Trained models are evaluated using unseen test data. Metrics such as accuracy, balanced accuracy, precision, recall, F1-score, confusion matrix, and ROC curves are used to provide a comprehensive assessment of detection effectiveness, accounting for class imbalance.

9. Result Generation and Analysis

The system generates classification results by labeling network traffic instances as normal or anomalous. Comparative analysis of ensemble models is performed based on evaluation metrics to identify the most effective classifier. The results support informed decision-making for network security monitoring and anomaly detection.

5.3 UML DIAGRAMS

Unified Modeling Language (UML) diagrams are standardized visual representations used to model, design, and document the structure and behavior of a software system. They help in clearly illustrating system components, user interactions, data flow, and execution processes in a graphical form. UML diagrams provide a common language for developers, designers, and stakeholders to understand system functionality. They are widely used to improve system analysis, design clarity, and communication throughout the software development process.

5.3.1 USE CASE DIAGRAM:

A use case diagram is a UML diagram that represents the functional requirements of a system by showing interactions between external actors and system use cases. It helps in understanding what the system does from the user's perspective. This diagram provides a high-level view of system functionality and user involvement.

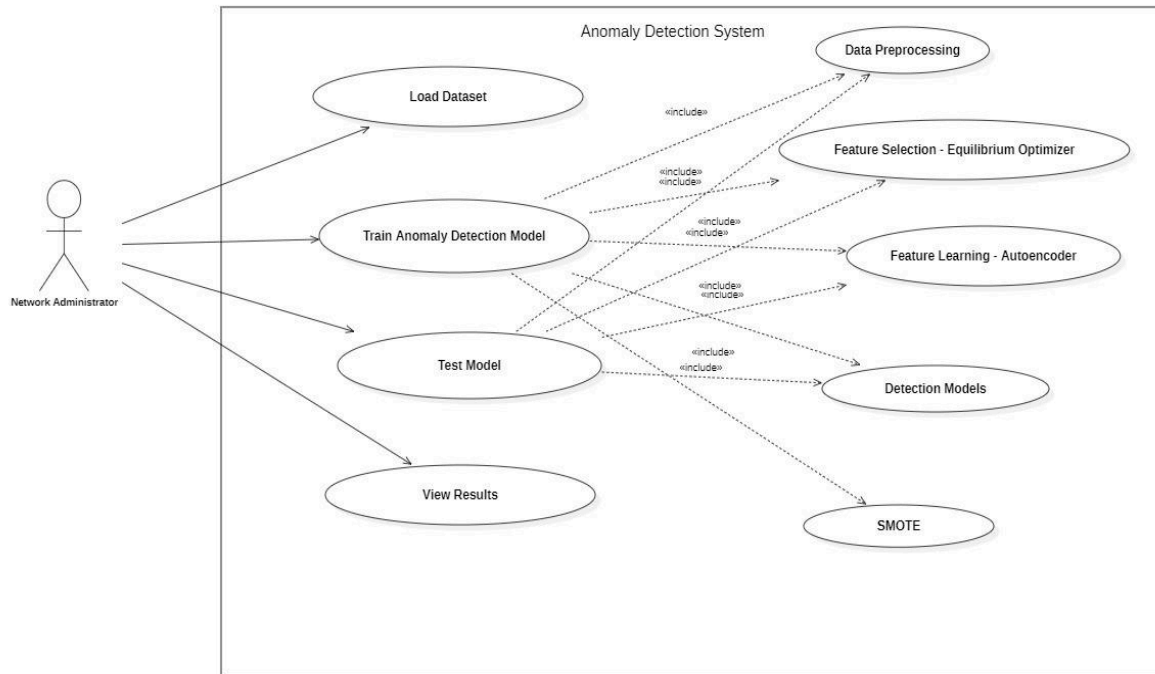


Fig 5.3.1 : Use Case Diagram

Actor : Network Administrator

The Network Administrator is the primary and only external actor interacting with the anomaly detection system. The administrator is responsible for managing the entire anomaly detection process, including dataset handling, model training, testing, and result analysis. The administrator does not directly perform internal machine learning operations, they initiate system functions and monitor outputs. This abstraction allows complex processing tasks to be executed automatically by the system while keeping user interaction simple and controlled.

Use Cases:

1. Load Dataset

This use case allows the Network Administrator to upload or load the UNSW-NB15 network traffic dataset into the system. The dataset serves as the primary input for both training and testing processes. Once loaded, the system verifies data integrity and prepares it for further processing. This use case is mandatory and must be completed before any training or testing operation can begin.

2. Train Anomaly Detection Model

This use case enables the Network Administrator to initiate the training of the anomaly detection model using the loaded dataset. During training, the system automatically executes multiple internal operations that are essential for building an accurate detection model. These internal operations are represented as include relationships in the use case diagram, indicating that they are compulsory steps of the training process.

Included Use Cases under Training:

a) Data Preprocessing

In this included use case, the system cleans and transforms the raw dataset by handling missing values, encoding categorical features, and removing irrelevant attributes. Preprocessing ensures the dataset is consistent and suitable for machine learning algorithms.

b) Feature Selection using Equilibrium Optimizer

This use case applies the Equilibrium Optimizer (EO) to identify the most relevant features that contribute significantly to anomaly detection. By selecting optimal features, the system reduces dimensionality, improves detection accuracy, and minimizes computational complexity.

c) Deep Feature Learning using Autoencoder

In this step, an autoencoder model is trained using the selected features to learn compact and meaningful latent representations. The encoder part of the autoencoder captures hidden patterns in network traffic data, enhancing the model's ability to distinguish between normal and anomalous behavior.

d) Class Imbalance Handling using SMOTE

This use case addresses the class imbalance problem in the dataset by applying SMOTE (Synthetic Minority Over-sampling Technique). SMOTE generates synthetic samples for minority attack classes, ensuring balanced training data and preventing biased learning.

e) Train Detection Models

Using the processed and balanced feature set, the system trains anomaly detection classifiers such as Extra Trees and CatBoost. These trained models learn decision boundaries for distinguishing normal traffic from anomalous traffic.

3. Test Anomaly Detection Model

This use case allows the Network Administrator to evaluate the performance of the trained model using unseen test data. During testing, the system applies the same preprocessing steps, EO-selected features, and trained autoencoder to the test dataset without retraining. The trained classifiers then generate predictions indicating whether the network traffic is normal or anomalous.

4. View Results

This use case enables the Network Administrator to view the classification outcomes produced by the detection models. The results include labels indicating normal or anomalous traffic. The system computes and displays performance metrics such as accuracy, precision, recall, F1-score, and ROC-AUC. These metrics help the administrator evaluate the effectiveness, reliability, and robustness of the anomaly detection system.

5.3.2 SEQUENCE DIAGRAM

A sequence diagram is a UML diagram that illustrates the time-ordered interaction between system components using lifelines and messages. It shows how data and control flow sequentially among objects to accomplish a specific task. This diagram helps in understanding the dynamic behavior of the system.

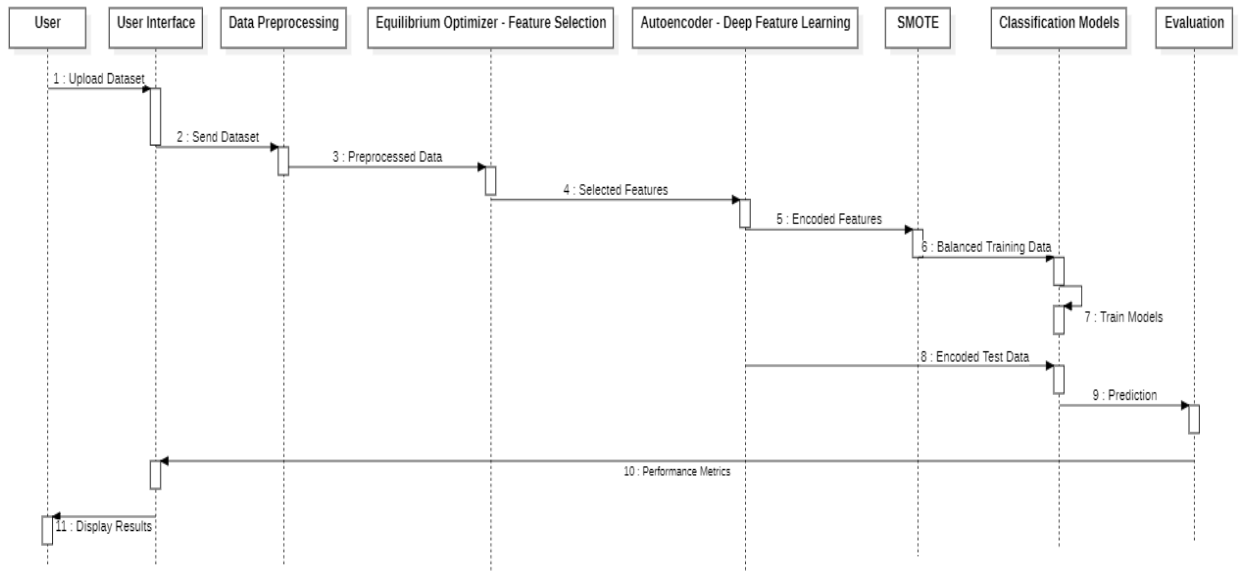


Fig 5.3.2 : Sequence Diagram

1. Lifelines

- **User** – Uploads the UNSW-NB15 dataset and views the final detection results.
- **User Interface (UI)** – Acts as the interaction layer between the user and the backend system, handling dataset submission and displaying performance metrics.
- **Data Preprocessing** – Cleans, transforms, and normalizes the raw network traffic data.
- **Equilibrium Optimizer** – Selects the most relevant features required for effective anomaly detection.
- **Autoencoder** – Performs deep feature learning and generates encoded feature representations.
- **SMOTE** – Balances the training dataset by generating synthetic samples for minority classes.
- **Classification Models** – Trains and tests anomaly detection models using the processed and balanced feature set.
- **Evaluation** – Computes performance metrics and produces evaluation results.

2. Interaction Flow

1. User → User Interface: Upload dataset

The user uploads the UNSW-NB15 dataset through the user interface to initiate the anomaly detection process.

2. User Interface → Data Preprocessing: Send dataset

The uploaded dataset is forwarded to the preprocessing module for cleaning and normalization.

3. Data Preprocessing → Equilibrium Optimizer Feature Selection: Send preprocessed data

The cleaned and scaled data is sent to the Equilibrium Optimizer to select the most relevant features.

4. Equilibrium Optimizer Feature Selection → Autoencoder: Send selected features

The optimal feature subset is forwarded to the autoencoder for deep feature learning.

5. Autoencoder → SMOTE: Send encoded features

The autoencoder generates deep encoded representations, which are passed to SMOTE for class balancing.

6. SMOTE → Classification Models: Send balanced training data

The balanced dataset is supplied to the classification models for training.

7. Classification Models → Classification Models (internal): Train models

Detection models such as Extra Trees and CatBoost are trained using the balanced feature set.

8. Autoencoder → Classification Models: Send encoded test data

Encoded test features are provided to the trained models for prediction.

9. Classification Models → Evaluation Module: Send predictions

The predicted anomaly labels are forwarded to the evaluation module.

10. Evaluation Module → User Interface: Return performance metrics

Evaluation metrics such as accuracy, precision, recall, and F1-score are sent to the user interface.

11. User Interface → User: Display results

The final detection results and performance metrics are displayed to the user.

5.3.3 ACTIVITY DIAGRAM

An activity diagram is a UML diagram that represents the workflow of a system by showing the sequence of activities and control flow from start to end. It helps in understanding how different operations are performed and in what order. Activity diagrams are mainly used to model the dynamic behavior of a system.

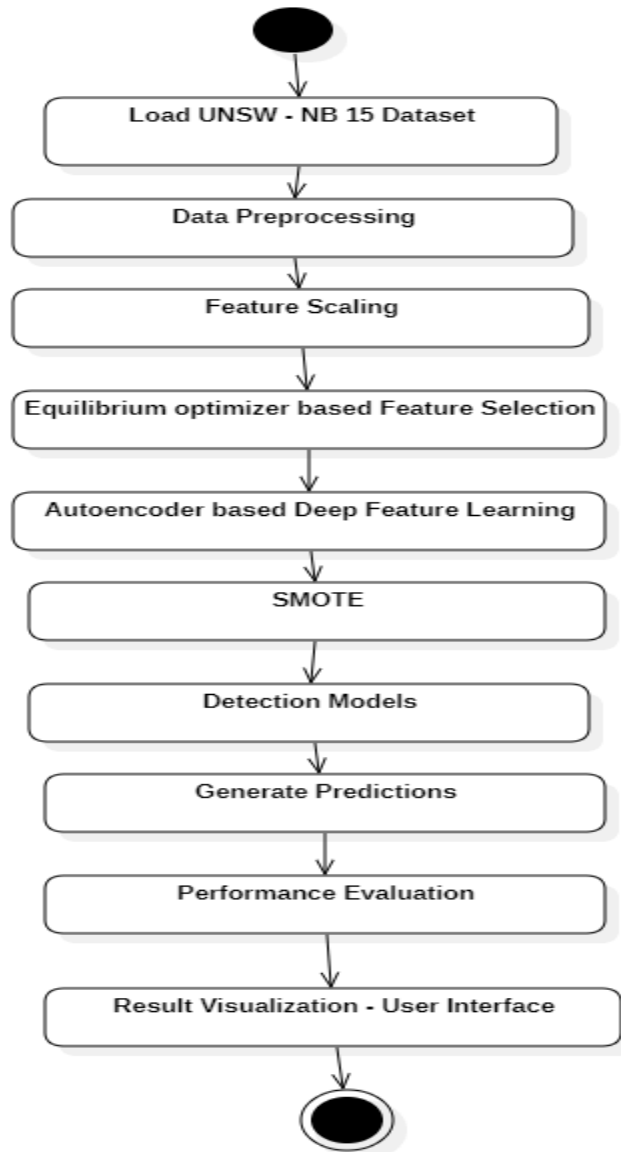


Fig 5.3.3 : Activity Diagram

1. Load UNSW-NB15 Dataset

This activity represents the initial step where the network traffic dataset is loaded into the system. The dataset contains labeled normal and attack records used for training and testing the anomaly detection model.

2. Data Preprocessing

In this activity, the raw dataset is cleaned and transformed by handling missing values, encoding categorical attributes, and removing irrelevant features. This step ensures the data is suitable for machine learning processing.

3. Feature Scaling

In this activity, feature scaling is carried out using the Standard Scaler. This ensures that all numerical features have a common scale and prevents features with larger magnitudes from dominating the learning process. Standardization improves the stability and performance of the anomaly detection models.

4. Equilibrium Optimizer–Based Feature Selection

This activity selects the most relevant features using the Equilibrium Optimizer algorithm. It reduces dimensionality, eliminates redundant features, and enhances detection accuracy.

5. Autoencoder-Based Deep Feature Learning

In this step, an autoencoder model learns compact and meaningful feature representations from the selected features. This helps capture complex patterns in network traffic data.

6. SMOTE

SMOTE balances the dataset by generating synthetic samples for minority attack classes. This prevents model bias toward majority classes and improves anomaly detection performance.

7. Detection Models

This activity involves training and testing classification models such as Extra Trees and CatBoost using the processed and balanced dataset to identify anomalous network behavior.

8. Generate Predictions

The trained models classify network traffic as normal or anomalous based on learned patterns. Prediction results are generated for evaluation.

9. Performance Evaluation

The system evaluates model performance using metrics such as accuracy, precision, recall, and F1-score. This helps assess the effectiveness of the proposed approach.

10. Result Visualization - User Interface

In the final activity, evaluation results and detection outcomes are presented. This aids in clear interpretation and analysis of system performance

5.4 SUMMARY

This chapter describes the system architecture of the proposed anomaly detection system and explains how different modules work together to process network traffic data. It outlines the major functional modules involved in preprocessing, feature selection, deep feature learning, anomaly detection, and evaluation. Additionally, UML diagrams are presented to visually represent the system structure, workflow, and interactions, providing a clear understanding of system behavior and design.

